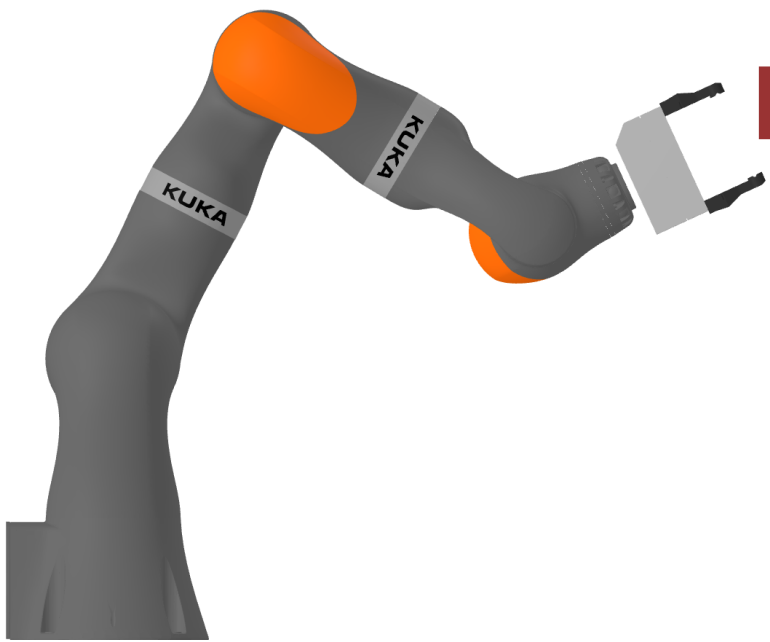


Programming Tasks

MIT 6.4210/2

Robotic Manipulation

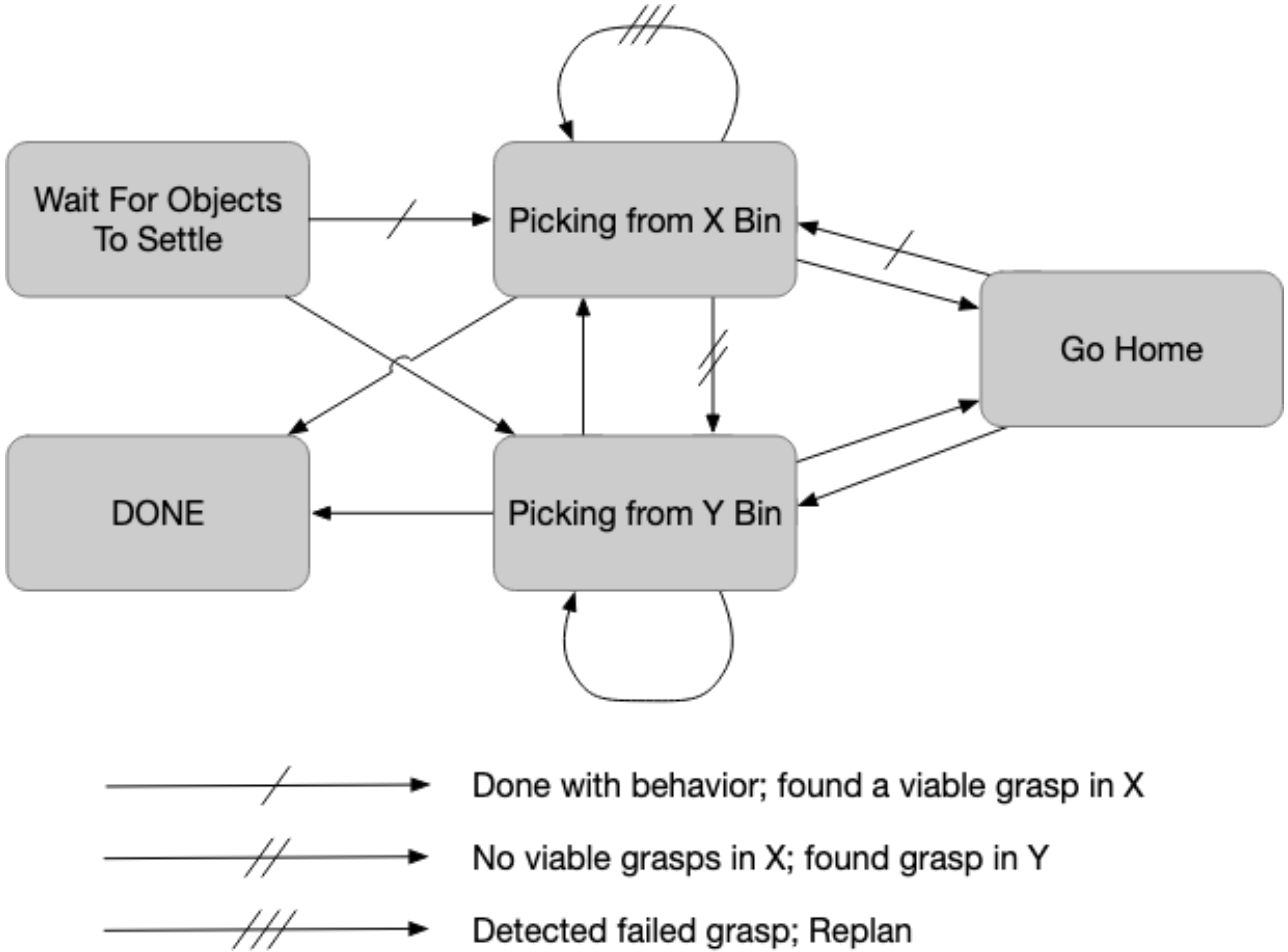
Fall 2025, Lecture 15

Massachusetts Institute of Technology

"Clutter clearing" (an early project at TRI)



https://people.csail.mit.edu/russt/uploads/clutter_clearing.html

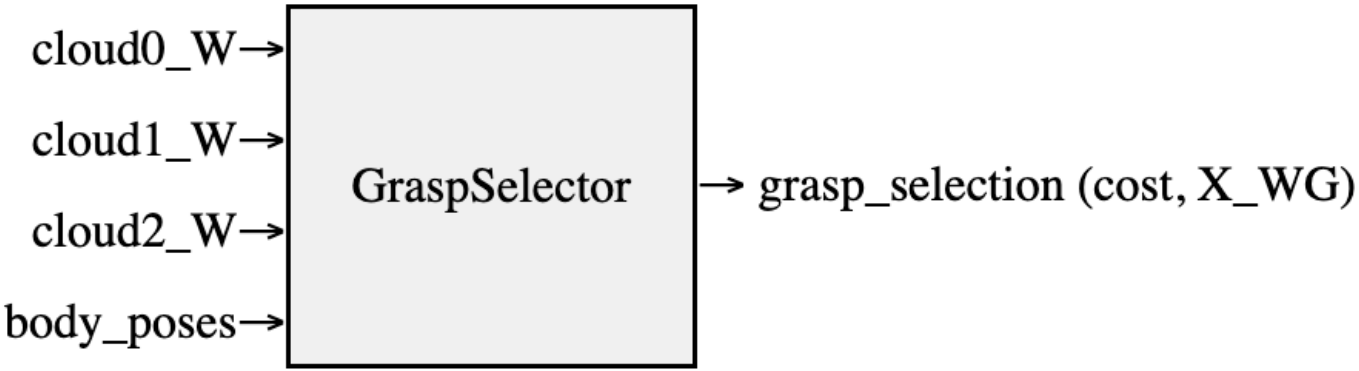

Wait For Objects To Settle
Picking from X Bin
Go Home
DONE
Picking from Y Bin
Done with behavior; found a viable grasp in X
No viable grasps in X; found grasp in Y
Detected failed grasp; Replan

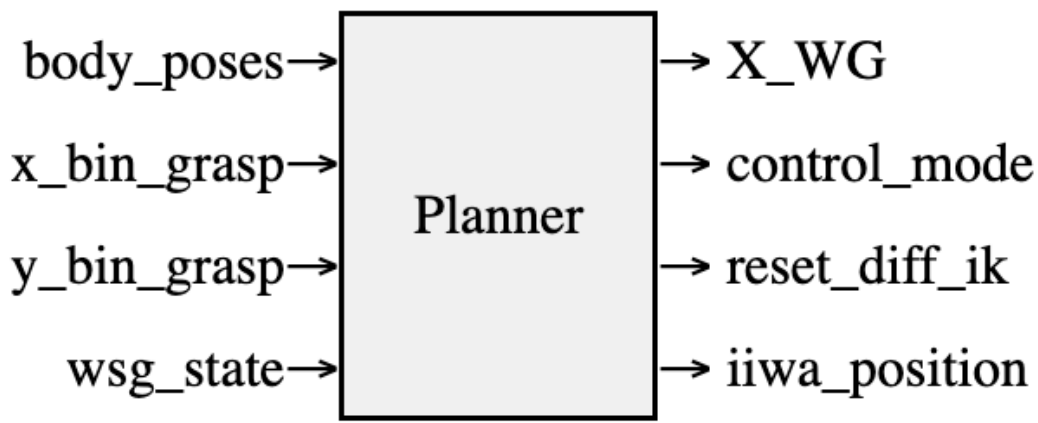
```
class PlannerState(Enum):
    WAIT_FOR_OBJECTS_TO_SETTLE = 1
    PICKING_FROM_X_BIN = 2
    PICKING_FROM_Y_BIN = 3
    GO_HOME = 4

class Planner(LeafSystem):
    def __init__(self, plant):
        LeafSystem.__init__(self)
        self.DeclarePeriodicUnrestrictedUpdateEvent(0.1, 0.0, self.Update)
        self._mode_index = self.DeclareAbstractState(
            AbstractValue.Make(PlannerState.WAIT_FOR_OBJECTS_TO_SETTLE))
                ...

    def Update(self, context, state):
        mode = context.get_abstract_state(int(self._mode_index)).get_value()
                ...
        state.get_mutable_abstract_state(
                int(self._mode_index)).set_value(PlannerState.PICKING_FROM_X_BIN)
```

```
class Planner(LeafSystem):
    def __init__(self, plant):
        ...
        self._traj_X_G_index = self.DeclareAbstractState(
            AbstractValue.Make(PiecewisePose()))
        self._traj_wsg_index = self.DeclareAbstractState(
            AbstractValue.Make(PiecewisePolynomial()))
        ...
```


cloud0_W
cloud1_W
cloud2_W
body_poses
GraspSelector
grasp_selection (cost, X_WG)


body_poses
x_bin_grasp
y_bin_grasp
wsg_state
Planner
X_WG
control_mode
reset_diff_ik
iiwa_position

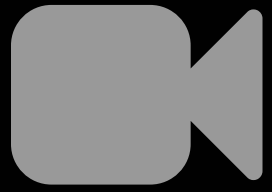
Failure modes of my simple state machine

I would group them into:

1. Motion planning. Collisions and DiffIK limitations.
2. Perception.
    - grasp far from object center of mass,
    - double-picks,
    - phantoms...
3. More than pick and place.
    - objects stuck in the corner
    - objects too big for the hand

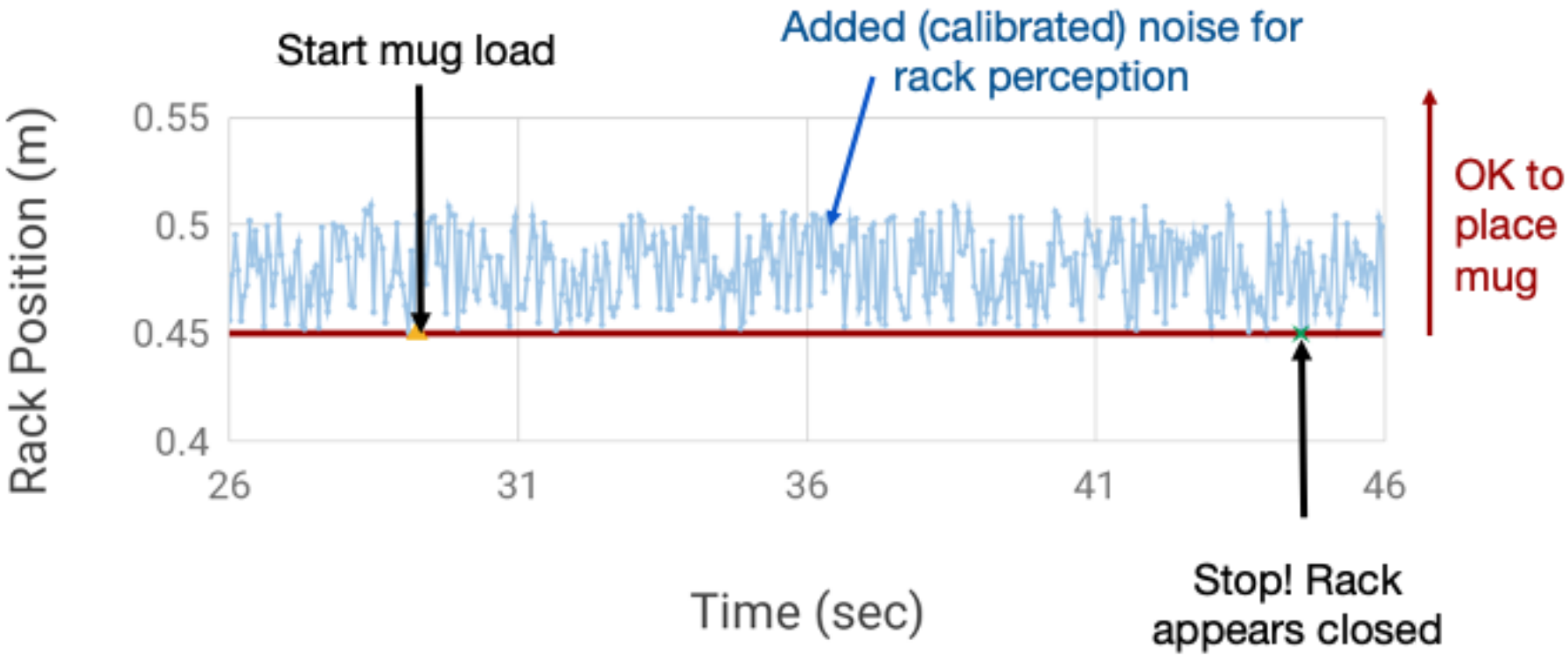
(Advanced) Full-stack integration testing

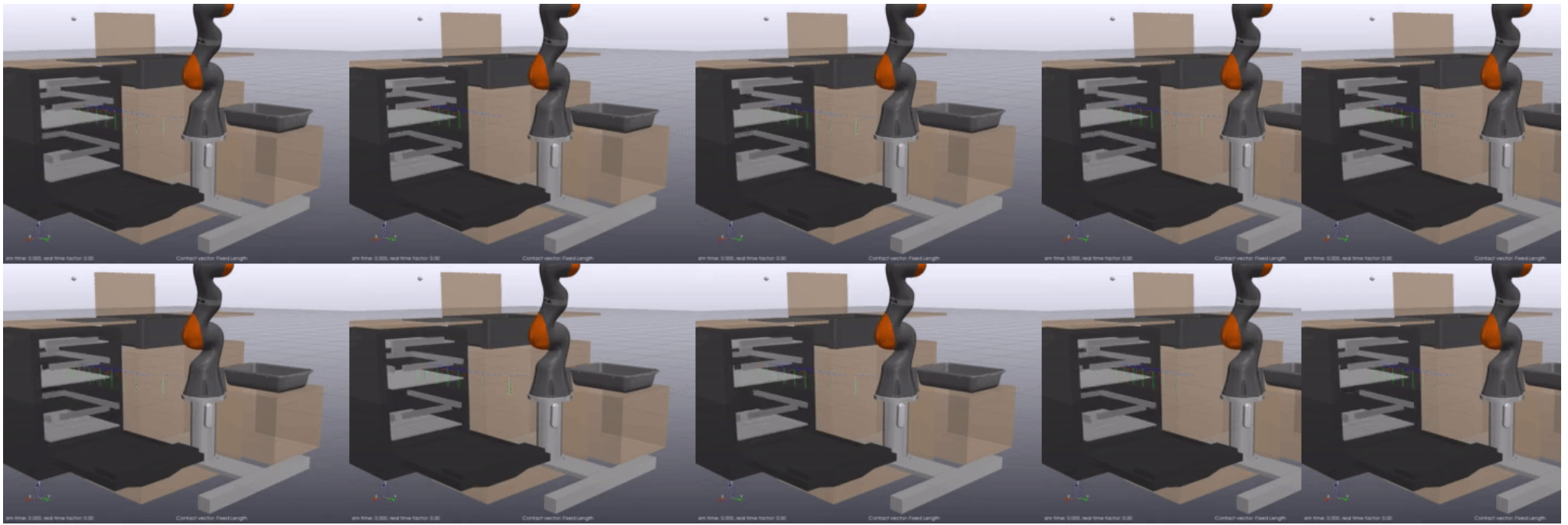




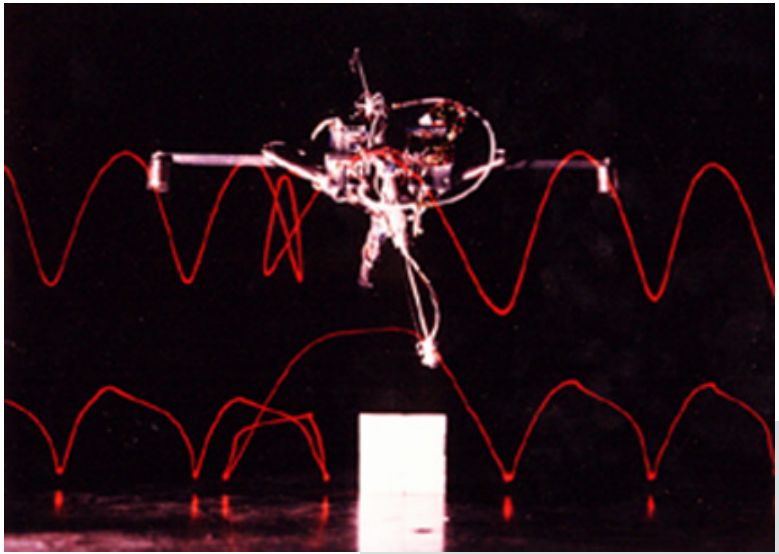
https://drake.mit.edu/doxygen_cxx/group__analysis.html#gae0a5d124aa451996b7a2727e52bf305e

Finding subtle bugs

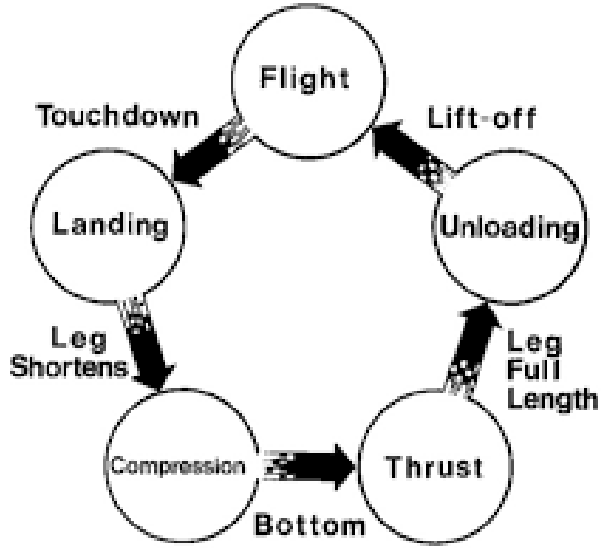


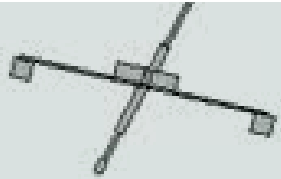


The Leg Lab Hopping Robots (CMU/MIT)



http://www.ai.mit.edu/projects/leglab/robots/robots.html







https://www.youtube.com/embed/aFuA50H9uek?enablejsapi=1&mute=1

https://wiki.ros.org/smach

Picking up a mug (original, hand-crafted version)

1. approach, s.t. mug is in view.
2. visual servo
3. insert to grasp
4. retract
5. move to pre-place
6. place (force termination)

work by Siyuan Feng et al. at TRI





Picking up a plate (original, hand-crafted version)

```
1. general approach the plate, s.t. it's roughly in view
for the wrist camera

2. visual servo for alignment

3. assuming known geometry and well alignment wrt the
plates, close the fingers to the right amount, insert one
finger tip between the plates.

4. push the top plate towards one side of the sink with
the finger tip. (terminate with force)

5. kind of scoop the gripper into a solid grasp, while
pushing onto the plate (terminate with force)

6. retract and go to pre-place configuration.

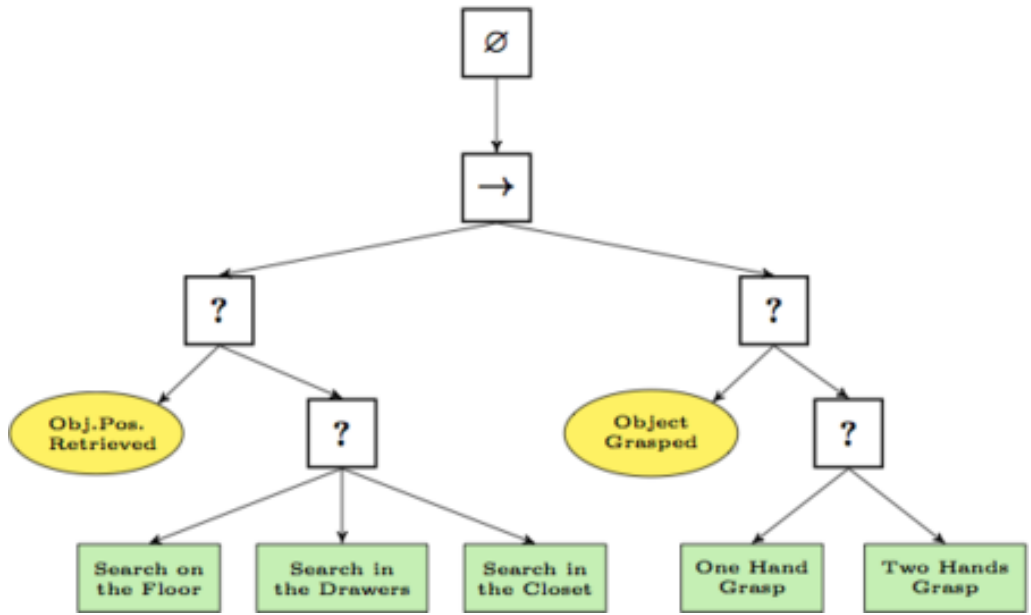
7. insert to place. (terminate with force)
```

work by Siyuan Feng et al. at TRI

Behavior Trees

Relative to state machines:

- Easier to compose
- Factored representation



https://docs.unrealengine.com/4.27/en-US/InteractiveExperiences/ArtificialIntelligence/BehaviorTrees/BehaviorTreeQuickStart/

https://docs.ros.org/en/kinetic/api/py_trees_ros/html/tutorials.html



https://github.com/RobotLocomotion/6-881-examples/blob/km_thesis_work/bt_planning/behavior_tree.py

Task planning

STRIPS problem definitions

https://en.wikipedia.org/wiki/Stanford_Research_Institute_Problem_Solver

- An initial state;
- The specification of the goal states – situations which the planner is trying to reach;
- A set of actions. For each action, the following are included:
  - preconditions (what must be established before the action is performed);
  - postconditions (what is established after the action is performed).

"Skill" Frameworks (aka "Motion Primitives")



```
/// Base type for all action primitives
class ActionPrimitiveInterface
{
  virtual bool IsCandidate(const State& state) const = 0;
  virtual Container GetOutcomes(const State& state) const = 0;
  virtual double EstimateActionCost(
      const State& current, const State& target) const = 0;
  virtual Container Execute(const State& state) = 0;
  virtual double Ranking() const = 0;
};
```



OpenDishwasherDoor
CloseDishwasherDoor
StartDishwasher
LoadSoapPacket
PullOutLowerRack
PullOutUpperRack
PullOutSilverwareRack
PushInLowerRack
PushInUpperRack
PushInSilverwareRack

PutPlateInDishwasher
ReorientMug
PutMugInDishwasher
OptimisticPutMugInDishwasher
PutSilverwareInDishwasher
PushUnmanipulableDish
SenseDishCounts
SenseDishToManipulate

work by Calder Phillips-Grafflin et al. at TRI




```
DishTaskState(const DishState& active_dish_state,
              int32_t clean_items_put_away, int32_t clean_items_in_dishwasher,
              int32_t dirty_items_in_dishwasher,
              int32_t dirty_items_available,
              const DishwasherState& current_dishwasher_state);

DishState(const RigidTransformd& dish_pose, DishType dish_type,
          DishProgress dish_progress, RackRequirement rack_required);

DishwasherState(bool known, bool door_open, bool lower_rack_out,
                bool upper_rack_out, bool silverware_rack_out,
                bool lower_rack_full, bool upper_rack_full,
                bool silverware_rack_full, bool started, bool soap_loaded);
```

```
enum class DishType : uint8_t {
  Unknown = 0x00,
  Mug = 0x01,
  Plate = 0x02,
  Silverware = 0x03
};

enum class RackRequirement : uint8_t {
  Unknown = 0x00,
  LowerRack = 0x01,
  UpperRack = 0x02,
  SilverwareRack = 0x03
};
```

```
enum class DishProgress : uint8_t {
  Unknown = 0x00,
  Unmanipulable = 0x01,
  LikelyManipulable = 0x02,
  Sensed = 0x03,
  OptimisticOriented = 0x04,
  Oriented = 0x05,
  Placed = 0x06,
  Released = 0x07
};
```

work by Calder Phillips-Grafflin et al. at TRI

We replan *before each action* to handle unexpected outcomes…



PDDL = Planning Domain Definition Language

Example: Gripper task with four balls

There is a robot that can move between two rooms and pick up or drop balls with either of his two arms. Initially, all balls and the robot are in the first room. We want the balls to be in the second room.

- *Objects*: The two rooms, four balls and two robot arms.
- *Predicates*: Is x a room? Is x a ball? Is ball x inside room y? Is robot arm x empty? [...]
- *Initial state*: All balls and the robot are in the first room. All robot arms are empty. [...]
- *Goal specification*: All balls must be in the second room.
- *Actions/Operators*: The robot can move between rooms, pick up a ball or drop a ball.

from https://www.cs.toronto.edu/~sheila/2542/s14/A1/introtopddl2.pdf

PDDL = Planning Domain Definition Language

```
(define (domain gripper-strips)
   (:predicates (room ?r)
                (ball ?b)
                (gripper ?g)
                (at-robby ?r)
                (at ?b ?r)
                (free ?g)
                (carry ?o ?g))

   (:action move
       :parameters  (?from ?to)
       :precondition (and  (room ?from) (room ?to) (at-robby ?from))
       :effect (and  (at-robby ?to)
                     (not (at-robby ?from))))



   (:action pick
       :parameters (?obj ?room ?gripper)
       :precondition  (and  (ball ?obj) (room ?room) (gripper ?gripper)
```

https://github.com/SoarGroup/Domains-Planning-Domain-Definition-Language/blob/master/pddl/gripper.pddl

PDDL = Planning Domain Definition Language

```
(define (problem strips-gripper4)
   (:domain gripper-strips)
   (:objects rooma roomb ball1 ball2 ball3 ball4 left right)
   (:init (room rooma)
          (room roomb)
          (ball ball1)
          (ball ball2)
          (ball ball3)
          (ball ball4)
          (gripper left)
          (gripper right)
          (at-robby rooma)
          (free left)
          (free right)
          (at ball1 rooma)
          (at ball2 rooma)
          (at ball3 rooma)
          (at ball4 rooma))
   (:goal (and (at ball1 roomb)
               (at ball2 roomb)
               (at ball3 roomb))))
```

https://github.com/SoarGroup/Domains-Planning-Domain-Definition-Language/blob/master/pddl/gripper-4.pddl

and then came the

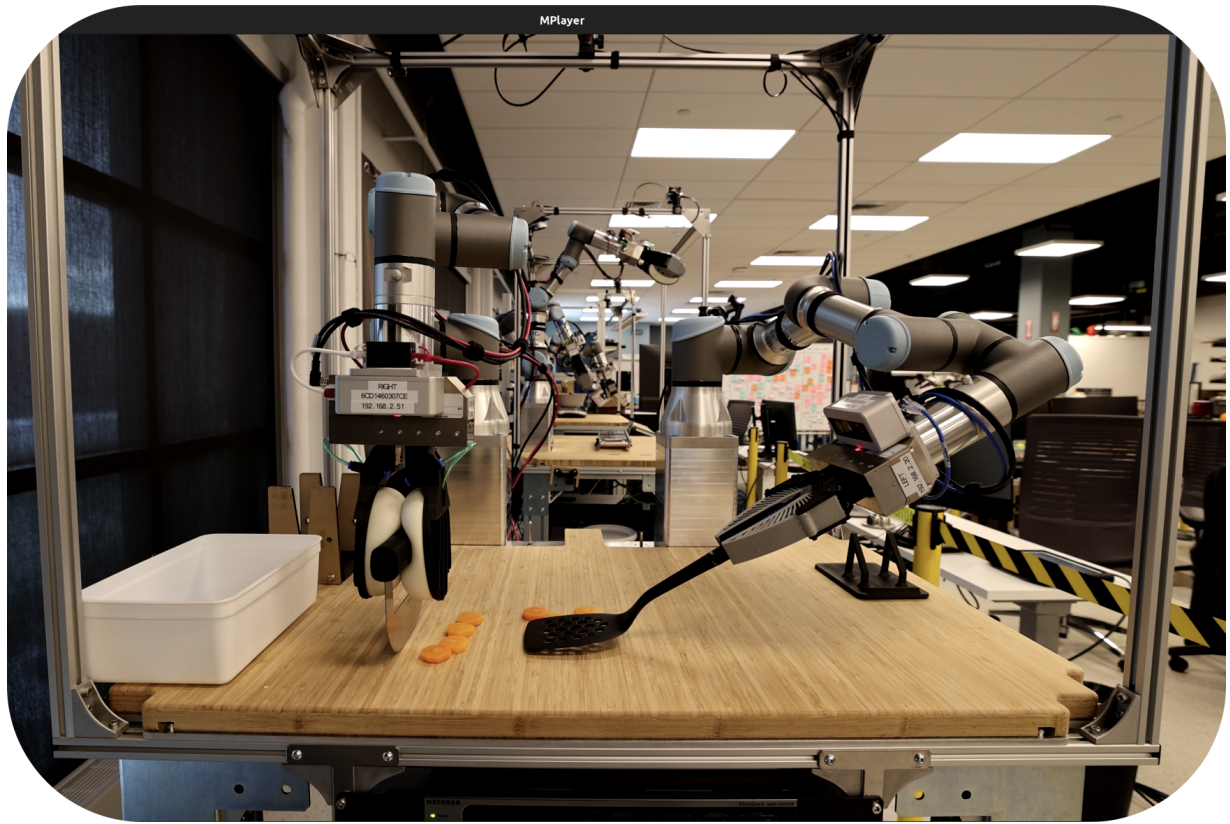
Large Language Models

Experiments with GPT-4 for task-level planning

Scenario:

Scooping vegetables

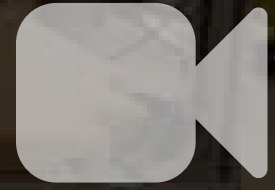
courtesy of Charlie Richter et al from TRI



TOYOTA RESEARCH INSTITUTE

my ChatGPT-4 session

Task Planning with Large Language Models



Still required: Symbol grounding

Perception system (or a human!) summarized environment state

The planner only reasoned about the *discrete* state / actions

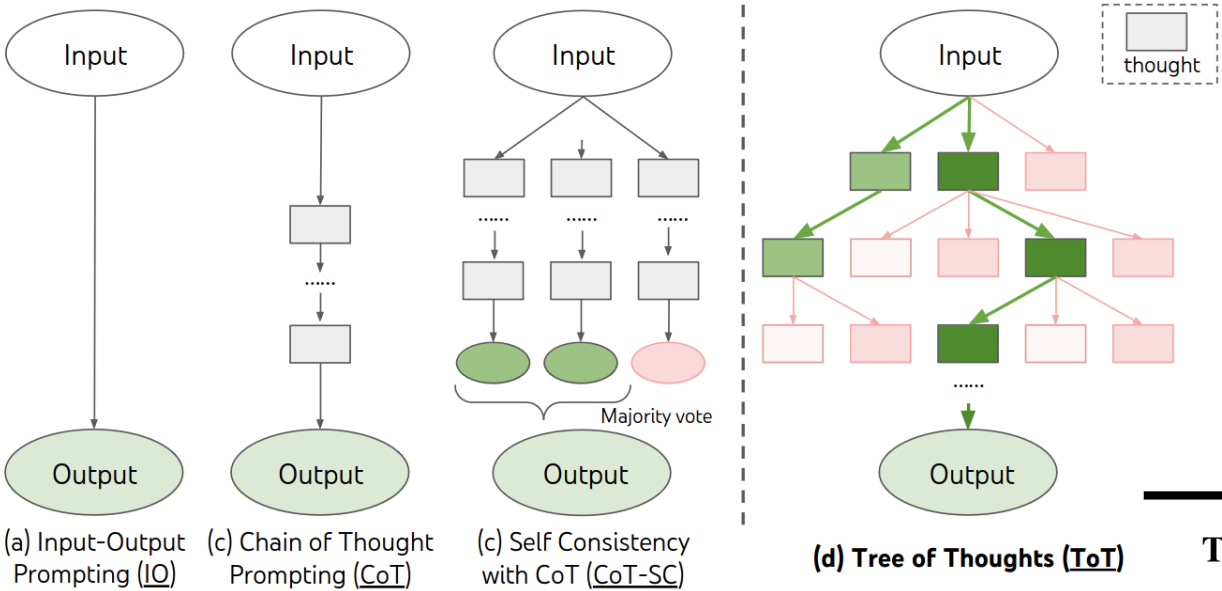
GPT-4V(ision)





I need to clean the whiteboard. Can you give me step-by-step instructions using these objects?

Long-term planning with LLMs



Tree of Thoughts: Deliberate Problem Solving with Large Language Models

Shunyu Yao Princeton University
Dian Yu Google DeepMind
Jeffrey Zhao Google DeepMind
Izhak Shafran Google DeepMind
Thomas L. Griffiths Princeton University
Yuan Cao Google DeepMind
Karthik Narasimhan Princeton University

Large Language Model

User

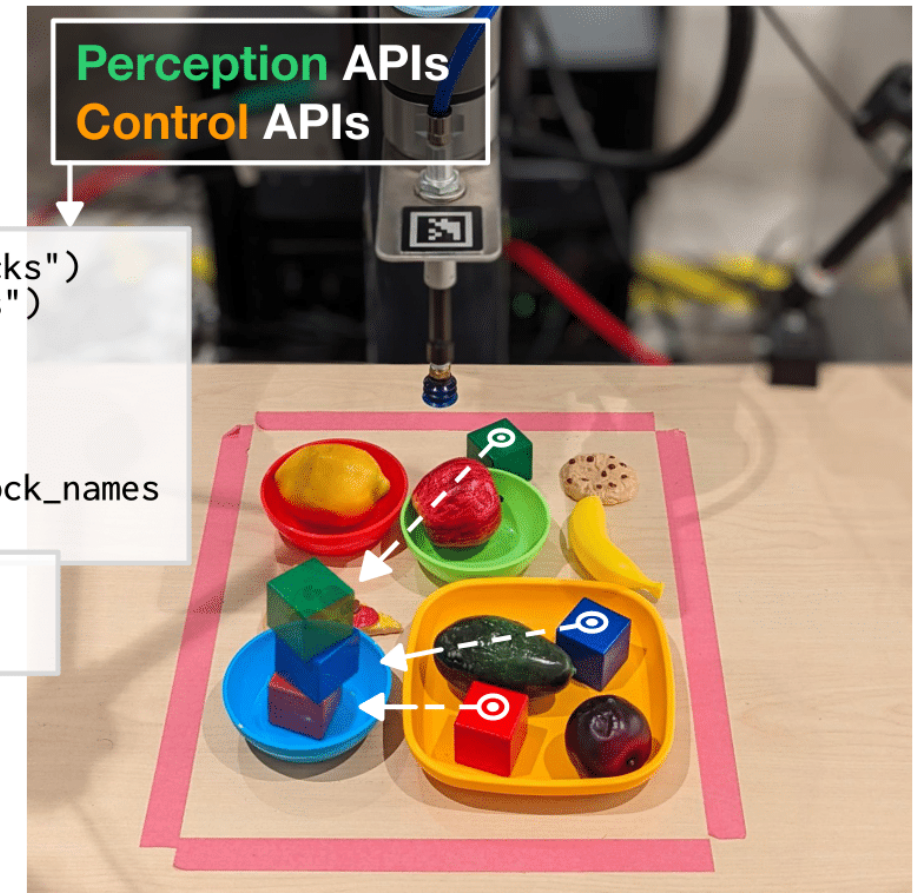
Stack the blocks on the empty bowl.

Policy Code

```
block_names = detect_objects("blocks")
bowl_names = detect_objects("bowls")
for bowl_name in bowl_names:
  if is_empty(bowl_name):
    empty_bowl = bowl_name
    break
objs_to_stack = [empty_bowl] + block_names
stack_objects(objs_to_stack)
```

```
def is_empty(name):
  ...
```

```
def stack_objects(obj_names):
  n_objs = len(obj_names)
  for i in range(n_objs - 1):
    obj0 = obj_names[i + 1]
    obj1 = obj_names[i]
    pick_place(obj0, obj1)
```



https://code-as-policies.github.io/

https://www.youtube.com/embed/1JDua3opFuM?enablejsapi=1

https://www.youtube.com/watch?v=1JDua3opFuM

Programming the Task Level

- Procedural code + message passing
- State machines / behavior trees
- Task planning (with online replanning)

- LLMs can potentially generate plans or simple policies; but it's not clear yet how far this will go